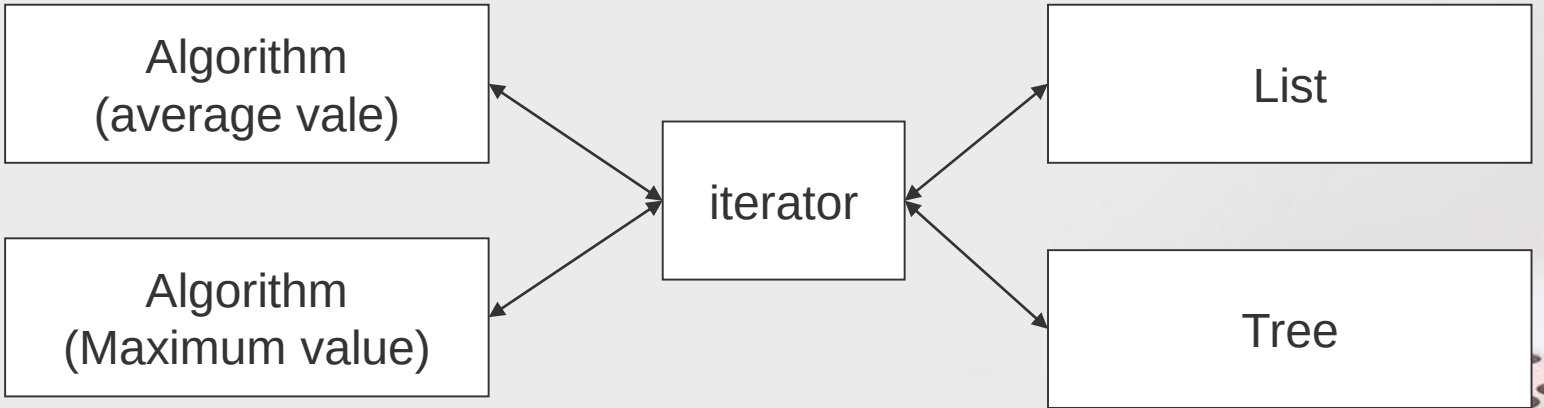
Data Structures

iterators

2017/2018

iterators

- Iterators are objects used to transverse through data structures.
  - They help to preserve the independence of the algorithms and data structures being manipulated.
  - The algorithms require an iterator, which is provided by the data structure.



iterators

- In java, iterators must implement the following interface:

```
public interface Iterator <E> {
   boolean hasNext();
   E next ();
   void remove ();
}
```

iterators

- The creation of a new data structure must be accompanied by the creation of a suitable iterator.

- The data structures provide appropriate iterators by a *factory* method, which creates and returns an iterator.

```
Iterator <E> iterator ();
```

- The interface *Iterable* <*T*> indicates the existence of the above-described method.

iterators

- Example: iterator factory

```
public class Pair <T> implements Iterable <T>
{
T p1, p2;

Iterator <T> iterator () {
return new IteratorPair <T> (this);
}
...
};
```

iterators

```
public class IteratorPar <T> implements Iterator <T> {
int counter = 0;
Pair <T> pair;
IteratorPar (Pair <T> p)   {p=pair;}

Boolean hasNext () {return counter = 2;!}

T next () {
    counter ++;
    switch (counter)
    {
          case 1: return pair.p1;
          case 2: return pair.p2;
          default: throw new NoSuchElementException ();
    };
}

void remove () {
    throw new UnsupportedOperationException ();
}
```

algorithms

- The algorithms can be made independent of the data containers through the use of iterators

```
public <T> boolean search (Iterable <T> m, T o) {
Iterator <T> it = m.iterator();
boolean next=it.hasNext();
while (next) {
    if (it.next () == o) // compares reference, not content
        return true;
    next=it.hasNext ();
   }
return false;
}
```

Iterators and Exceptions

- Iterators can / should generate the following exceptions:
  - **UnsupportedOperationException** - The operation (eg: removal) is not supported.
  - **NoSuchElementException** - Attempt to access an element that does not exist
  - **IllegalStateException** - removal attempt without advancing to the first element or attempt to remove the same element more than once.
  - **ConcurrentModificationException** - When attempting to use an invalid iterator. An iterator is invalid when the collection was changed externally (by another iterator, for example).